UNIVERSIDAD DE LAS FUERZAS ARMADAS
“ESPE”

INGENIERÍA DE SOFTWARE

Algoritmos de Detección de Vulnerabilidades

en Código Generado con Inteligencia Artificial

Integrantes:
Josué Chango
Yostin Sisalema

Docente: Ing. Rea Guaman Angel Marcelo

Pregunta de Investigación

¿Qué algoritmos permiten detectar vulnerabilidades en código fuente generado por herramientas de IA

Contexto

Herramientas como GitHub Copilot, Cursor y CodeWhisperer generan código que puede replicar patrones inseguros aprendidos de repositorios con deuda técnica.

Problema

Los LLM (large languaje model) tienen mayor recall para detectar vulnerabilidades, pero también generan más ruido. Las ASAT (Automated Security Analysis Tools) son precisas pero no capturan patrones nuevos.

Alcance

CWE Top-25: inyección SQL, XSS, buffer overflow, aleatoriedad débil. Código en C/C++, Java y lenguajes soportados por Copilot.

ISO/IEC 25010:2023 — SQuaRE

Por qué aplica a nuestro tema?

La ISO/IEC 25010:2023 define el modelo de calidad de producto de software en 9 características. La característica **Security** incluye la subcaracterística Resistance (nueva en 2023), que mide la capacidad del software de resistir ataques, incluyendo vulnerabilidades introducidas durante la generación automática de código.

ISO/IEC 25010:2023 · iso.org/standard/78176.html

Subcaracterísticas de Security

- Confidentiality
- Authenticity
- Integrity
- **Resistance**
- Non-repudiation
- Accountability

Conexión con la investigación: los algoritmos de detección (Q1) son el mecanismo técnico para garantizar la subcaracterística Resistance en código generado por IA.

Criterios de Inclusión

Temporalidad

Publicaciones entre 2019 y 2026. Se priorizan papers de 2023-2026 para capturar los avances más recientes en LLMs aplicados a detección de vulnerabilidades.

Relevancia temática

Trabajos que aborden explícitamente detección automática de vulnerabilidades en código fuente usando ML, DL o LLM. Se incluyen surveys si comparan ≥5 técnicas.

Métricas reportadas

El paper debe reportar al menos una métrica cuantitativa: accuracy, precision, recall o F1-Score sobre un dataset de referencia (NVD, SARD, Devign, BigVul, CVEfixes).

Base de datos indexada

Solo fuentes indizadas en: IEEE Xplore, ACM Digital Library, ScienceDirect (Elsevier), Nature/Springer, arXiv (cs.CR/cs.SE), MDPI. No se admiten blogs ni grey literature.

Lenguajes soportados

Estudios sobre C/C++, Java, Python o lenguajes compatibles con herramientas de generación de código IA (GitHub Copilot, CodeWhisperer, Cursor).

Acceso completo

Texto completo disponible para descarga: open access, acceso institucional ESPE o versión preprint en arXiv/ResearchGate. No se usan solo abstracts.

Criterios de Exclusión

Antes de 2019

Publicaciones anteriores a 2019 se excluyen por no reflejar el estado actual de los modelos Transformer (BERT, GPT) aplicados a código.

Seguridad en red / infraestructura

Trabajos enfocados en detección de intrusos (IDS), seguridad de red, malware en binarios o firmware. El alcance es exclusivamente código fuente.

Artículos duplicados / retractados

Se elimina la versión más antigua cuando existe una versión actualizada del mismo trabajo. También se excluyen artículos retractados por las editoriales.

Sin evaluación empírica

Se descartan propuestas teóricas, capítulos de libro o artículos que no presenten experimentos cuantitativos sobre datasets de vulnerabilidades.

Solo análisis dinámico

Estudios basados únicamente en fuzzing, taint analysis en ejecución o análisis dinámico sin componente de análisis de código fuente estático.

Idioma distinto al inglés o español

Artículos publicados en otros idiomas que no cuenten con abstract completo en inglés, por limitaciones de revisión y reproducibilidad.

Fuentes Seleccionadas

Hussain et al. (2024)

Vulnerability detection in Java source code using QCNN with self-attentive pooling

Nature / Scientific Reports · 10.1038/s41598-024-56871-z
Accuracy 99.2% — Java · AST+CFG+PDG+CodeBERT+QCNN

Huang & Aumpansub (2024)

Vulnerability Detection in C/C++ Code with Deep Learning (BGRU)

arXiv:2405.12384v3 · arxiv.org/abs/2405.12384
Accuracy 92.5% — C/C++ · Program slices · BGRU+ADAM

Zhu et al. (2026)

BMGANet: Deep learning for vulnerability detection via token & function-level features

Computers and Electrical Engineering (Elsevier) · 10.1016/j.compeleceng.2026.110999
F1 89.19% promedio — C/C++ · BERT+LSTM+Attention+PDG

Zou et al. (2025)

Code vulnerability detection based on AUG-PDG and optimized CodeBERT

Nature / Scientific Reports · 10.1038/s41598-025-23029-4
Reduce ruido con gestión de ámbito de variables — AUG-PDG

Liang et al. (2024)

Survey of source code vulnerability analysis based on deep learning

Computers & Security (ScienceDirect) · sciencedirect.com/science/article/pii/S0167404824004036

Survey: clasifica y compara DL, ML clásico y LLM para detección

Varios autores (2025)

LLMs vs. Static Code Analysis Tools: A Systematic Benchmark

arXiv:2508.04448 · arxiv.org/html/2508.04448v1
Benchmark 63 vulns C# — LLM F1≈0.75-0.80 vs ASAT F1≈0.26-0.55

Tres Familias Algorítmicas — Síntesis

¿Qué algoritmos detectan vulns en código IA?

Análisis Estático (ASAT)

- Reglas predefinidas
- Ejecución simbólica
- SonarQube · CodeQL
- Snyk · Flawfinder

Rápido, sin enviar código

✘ No detecta patrones nuevos

LLMs como Detectores

- GPT-4.1 · Mistral · DeepSeek
- CodeGemma · CodeBERT
- Mayor recall (F1 ≈ 0.75-0.80)
- Benchmark: 63 vulns C#

Explicaciones naturales

✘ Falsos positivos + alucinaciones

Deep Learning sobre Código

- LSTM · GRU · BiLSTM · GCN
- AST, CFG, PDG (grafos)
- CodeBERT · BERT + LSTM

Detecta patrones no definidos

Tendencia 2024-2026: combinar ASAT + DL + LLM. ASAT en pipeline CI/CD · LLM para revisiones de seguridad profundas (ISO/IEC 25010:2023 — Resistance)

GRACIAS

Yostin Sisalema · Josué Chango · Ing. Rea Guaman A.M. · ESPE NRC 30821 · Abril 2026